Does Deterministic Generate–Validate–Repair Reduce Unsafe LLM-Generated Network Changes? A Confirmatory Paired Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a sealed paired confirmatory study (n = 200 intent→configuration tasks) to test whether a deterministic generate–validate–repair (GVR) pipeline that consults a deterministic NetOps validator reduces validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline. The run used a single hosted model (gpt-5-mini) with adapter-enforced shared-initial generation semantics and a primary deterministic validator (deterministic_netops_validator_v5). Execution completed all planned pairs (completed_pairs = 200) consuming 200 model calls (one shared initial generation per task); no repair calls were issued because every initial candidate passed the validator. Paired contingency: n11 = 200, n10 = 0, n01 = 0, n00 = 0; estimate = 0.0; exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.0, 0.0] (10,000 resamples, seed = 1729). Because the baseline validator-detected unsafe incidence was 0% on this task bank, the preregistered >=50% relative-reduction hypothesis could not be meaningfully evaluated. We report the sealed design, execution accounting (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z), the Mininet 10% holdout (20 tasks) that produced zero validator–emulator disagreements, and an artifact-grounded discussion of operational implications and threats to validity (preregistration id: cnsms2026-gvr-effectiveness-02).**

I. Introduction

Automating intent→configuration translation with generative language models can increase NetOps productivity but risks producing incorrect or unsafe changes that cause outages or policy violations. Prior work therefore proposes grounding, deterministic validation, and repair loops (generate–validate–repair, GVR) to detect and correct unsafe candidates before application. The operational benefit of automated repair is conditional on three factors: (i) baseline prevalence of validator-detectable failures from the chosen generator/prompting policy; (ii) validator coverage and fidelity; and (iii) the available repair budget and repair-prompt effectiveness. We preregistered a narrowly scoped confirmatory question: Does an automated, deterministic GVR pipeline that consults a deterministic NetOps validator materially reduce validator-detected unsafe or semantically incorrect network changes relative to an LLM-only generation pipeline on a curated 200-task intent→configuration bank? The study was designed as a paired experiment that fixes the generator output per task and isolates the marginal effect of invoking a validator+repair on the same candidate.

II. Related Work

Surveys and empirical studies across LLM applications document factual errors, hallucinations, prompt sensitivity, and non-determinism; mitigation approaches include retrieval grounding, verifier-assisted repair, and constrained agent architectures. NetOps prototypes show feasibility of intent→configuration generation but typically lack instrumented, production-like failure-rate evaluations. Architectural proposals recommend combining LLM generation with deterministic validators (e.g., Batfish-style checkers) and with guarded agent patterns; red-team audits demonstrate configuration-level brittleness and tool-description injection vectors that can alter safety outcomes. Our study positions itself as an externally auditable, preregistered measurement that isolates the marginal effect of deterministic repair for a fixed model and sampling policy, addressing gaps about when GVR actually reduces validator-detected failures in practice.

III. Methodology

Preregistration and inferential plan. The study preregistration (cnsms2026-gvr-effectiveness-02) fixed a paired design with n = 200 tasks, one shared deterministic initial generation per task, and a repair budget of at most one automated repair call per task. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline; the primary test was an exact two-sided McNemar test ($\alpha = 0.05$). Confidence intervals were computed via bootstrap percentile (10,000 resamples, bootstrap_seed = 1729). Power planning assumed a baseline unsafe incidence ≈30% and a targeted 50% relative reduction; these assumptions were recorded in the preregistration and used only to justify sample size. The analysis executor was paired_binary_analysis_v1.

Task bank, validators, and holdout. We used a curated, machine-checkable intent→configuration task bank with 200 tasks across four topology clusters (enterprise, campus, datacenter, edge; 50 tasks each). Each task encodes an initial and expected final state, workflow constraints (make-before-break, must_be_down_for_fields, group-active minima), and a required_sequence. The primary deterministic checker (deterministic_netops_validator_v5) enforces syntactic and intent/constraint checks and emits structured diagnostics (valid flag, violation codes, parsed_command_count, required_command_count). A Mininet-based data-plane emulator was preregistered as a secondary disagreement detector applied to a randomized 10% holdout (20 tasks) to detect validator–emulator mismatches; execution produced an analysis/contamination_summary.csv artifact recording zero disagreement flags for that holdout.

Adapter semantics, shared-initial design, and causal isolation. The hosted_netops_gvr_v1 adapter enforced shared_initial generation semantics: for each task the system produced a single deterministic initial candidate that was reused for both baseline (LLM-only) and guarded (GVR) conditions. This design choice intentionally removes sampling variance across conditions so the paired comparison isolates the marginal effect of invoking the validator and (if needed) issuing a single repair prompt on the identical generator output. Adapter accounting (deterministic_reconciliation artifacts) records hosted_model_call_event_count = 200 and stage_counts = {shared_initial_generation: 200}, confirming single shared invocations per task. By construction, any benefit attributable to repeated or stochastic re-sampling is excluded from the primary estimand; this is a preplanned trade-off to obtain a controlled causal contrast between (A) applying the generator output unchecked and (B) validating–and possibly repairing–that same output.

Repair policy, budgeting, and handling. The preregistered repair policy allowed at most one automated repair prompt to the same LLM when the primary validator reported violations. Repair prompts (when issued) and any repaired candidate were re-validated by deterministic_netops_validator_v5 and recorded in validator_trace fields (repair_applied, repair_prompt_sha256, final_configuration). The execution contract limited maximum model calls to 400 (200 initial + up to 200 repairs); the adapter enforced this cap. Missingness/exclusion rules deterministically excluded pairs where both conditions failed due to infrastructure execution failures; tasks flagged for validator–emulator disagreement were excluded from the primary analysis per the contamination_plan, with deterministic sensitivity analyses preregistered for alternative treatments.

Randomization, seeds, and reproducibility artifacts. Task ordering was randomized with a fixed generation seed as recorded in the task_manifest entries (example fields: generation_seed). Analysis used a paired, shared-initial seed schedule and the bootstrap CI procedure used bootstrap_seed = 1729 with 10,000 resamples (analysis/analysis_log.jsonl captures these parameters). The Mininet 10% holdout executed as preregistered and is summarized in analysis/contamination_summary.csv (20 tasks; zero disagreement flags). We disclose a documented reproducibility gap: the explicit labeled randomized selection seed identifying which tasks comprised the 10% Mininet holdout is not present in the labeled artifacts, preventing exact deterministic reconstruction of the drawn holdout despite the contamination summary reporting zero disagreements.

Statistical implementation and exact tests. The primary confirmatory test was implemented as an exact McNemar two-sided test on the 2×2 paired table (n11, n10, n01, n00) produced by paired_binary_analysis_v1. Where the discordant counts were zero or small, exact inference is necessary; bootsrapped paired-difference percentile CIs were computed to reflect the empirical paired distribution under resampling (10,000 resamples; seed recorded). Secondary/exploratory analyses (preplanned) included logistic regressions (repair_success ~ topology_cluster + initial_validator_failure_type + repair_iterations) and descriptive summaries of repair iteration counts and model-call costs; these remained exploratory and were executed only when applicable data were generated.

Contamination detection and conservative primary analysis rule. The preregistered contamination plan used the Mininet emulator as an independent end-to-end check for a randomized holdout. Any pair where Batfish-style validator verdict != Mininet verdict would be flagged and excluded from the primary confirmatory analysis to conservatively avoid reliance on a possibly incomplete primary validator. The analysis artifact analysis/contamination_summary.csv shows zero flags for the 20-task holdout; deterministic_reconciliation.json confirms pair_accounting_consistent = true. Because no contamination flags occurred, the preplanned conservative exclusion branch was not activated.

Execution accounting and archived artifacts. The run executed autonomously under the frozen plan (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z). Model calls used = 200 (one shared initial generation per task); repair calls used = 0. Core archived artifacts relevant to reproducing and auditing the methodology include execution/raw_results.jsonl, execution/model_configuration.json, execution/task_manifest.jsonl, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, analysis/contamination_summary.csv, analysis/analysis_log.jsonl, and deterministic_reconciliation.json. These artifacts capture adapter accounting, validator traces (validation_before/after objects with parsed_command_count, required_command_count, violation_codes), and the bootstrap parameters. All handling decisions (exclusions, missingness treatment) were deterministic and logged in analysis/analysis_log.jsonl.

IV. Results

Execution outcomes. The run completed all planned episodes: completed_episode_count = 400 (shared initial generation reused across conditions) and complete_pair_count = 200. Adapter accounting shows model_calls_used = 200 (one shared initial generation per task) and cache_miss_count = 200. No repair calls were issued because every initial candidate passed deterministic_netops_validator_v5. Condition summaries (analysis/condition_summary.csv) show baseline successes = 200/200 and guarded successes = 200/200 (success_rate = 1.0 for both). The paired contingency (analysis/paired_contingency_table.csv) is n11 = 200, n10 = 0, n01 = 0, n00 = 0.

Primary inferential analysis. With zero discordant pairs the observed paired difference estimate = 0.0. The exact McNemar two-sided p = 1.0. The bootstrap percentile 95% CI (10,000 resamples; seed = 1729) is [0.0, 0.0]. Under the preregistered estimand and test the prespecified >=50% relative-reduction hypothesis is logically untestable on this sample because there were no baseline validator failures to reduce.

Representative diagnostics. Representative per-task artifacts (for example, pair-000001) confirm that the shared initial generator outputs matched the task required_sequence and that validator_trace.validation_after.valid = true with violation_count = 0. Difficulty annotations in validator traces (examples: make_before_break_uplink_switchover, paired_link_atomic_mtu_change) and parsed_command_count/required_command_count show that tasks exercised medium-to-very_hard workflow constraints despite generating validator-satisfying candidates. The Mininet 10% holdout (20 tasks) executed as preregistered; analysis/contamination_summary.csv reports zero validator–emulator disagreements, and deterministic reconciliation artifacts report all consistency checks passed (pair_accounting_consistent = true).

Unexecuted branches. Planned sensitivity branches for flagged contamination treatments and alternative missingness handling were not invoked because no tasks were flagged and no episodes failed. Secondary exploratory estimands (average repair iterations, GVR-introduced failure rate, model-call cost per successful repair) are empty in the analysis bundle because no repair attempts occurred; the analysis artifacts record these secondary_results as empty.

V. Discussion

Conditional interpretation. The degenerate null outcome demonstrates a logical constraint: when the chosen generator, prompt template, and sampling policy produce validator-passing candidates for every item in a workload, an added validator-coupled repair step cannot reduce validator-detected failures. This observation is artifact-grounded and operationally important: the marginal value of automated repair depends on baseline validator failure prevalence and on validator coverage.

Why baseline failures were zero (artifact-based explanations). First, the prompt template and enforced shared_initial semantics produced outputs that conformed to explicit task-bank constraints. Second, the task bank is curated with clear required_sequences and machine-checkable constraints that map directly to validator checks—this reduces semantic ambiguity and makes correct outputs easier to generate with a deterministic prompt policy. Third, the primary validator enforces the task-encoded constraints but may omit device-level idiosyncrasies detectable only by an end-to-end emulator or real device; the recorded Mininet holdout produced zero disagreements but the run also discloses a missing labeled selection seed for that holdout (see reproducibility note). Fourth, the single repair budget and shared_initial design remove per-condition re-sampling as a pathway to obtain alternative repairable candidates.

Operational implications. Before deploying automated repair, practitioners should measure baseline validator failure prevalence on representative operational workloads and examine validator coverage with end-to-end emulation. When baseline validator failures are rare under the chosen generator/prompt policy, investing in richer validator fidelity (vendor semantics, device models), multi-sample generation, multi-model ensembles, or targeted adversarial testing may yield higher safety returns than automated repair. Conversely, when baseline failures are non-negligible, a well-engineered GVR pipeline with sufficient repair budget can reduce validator-detected issues; realized gains will depend on repair-prompt quality, budget, and validator completeness.

Threats to validity. Internal validity: preregistration, deterministic adapter enforcement, and frozen stopping rules minimize post-hoc choices, but the shared_initial design intentionally trades sampling generality for a controlled paired comparison. Construct validity: the primary validator enforces explicit task constraints but may not model all real device behaviors; emulator disagreements would reveal such blind spots but none were observed on the 20-task Mininet holdout. External validity: a single hosted model (gpt-5-mini), single prompt/sampling policy (shared initial deterministic sampling), a curated synthetic/public task bank, and a single repair budget were used; results may differ with other models, nonzero temperature sampling, multiple samples per condition, adversarial workloads, or production device heterogeneity. Conclusion validity: with zero discordant pairs the McNemar test is uninformative for the preregistered effect size; nonetheless the observed zero-discordance is a reproducible operational datum documented in the archived artifacts.

Reproducibility note and documented gap. All execution artifacts (model-call logs, validator traces, paired contingency tables, bootstrap parameters) are archived in the execution bundle. The Mininet holdout evidence is captured in analysis/contamination_summary.csv which reports zero disagreements for the 20-task holdout; this artifact demonstrates that the secondary emulator check executed and found no validator–emulator mismatches. A remaining reproducibility gap is that the explicit randomized selection seed for the Mininet holdout is not labeled in the archived artifacts, preventing exact deterministic reconstruction of which tasks were drawn into the holdout despite the contamination summary reporting zero disagreements. We disclose this gap transparently in the Disclosure Statement.

VI. Limitations

- Task-bank scope: the confirmatory sample used a curated synthetic/public intent→configuration bank (n = 200); results therefore reflect this bank and may not generalize to broader production workloads or adversarial distributions.
- Single-model and shared-initial setting: only gpt-5-mini (hosted) with adapter-enforced shared-initial generation was tested (execution artifacts record hosted_model_call_event_count = 200 and stage_counts

shared_initial_generation = 200). Outcomes may differ across model families, independent per-condition sampling, nonzero temperature sampling, or larger repair budgets.

- Validator coverage and oracle limitations: the primary deterministic validator enforces a defined set of syntax/intent constraints; validators can fail to capture real device idiosyncrasies, vendor-specific behaviors, or emergent failure modes detectable only by end-to-end deployment.
- Adversarial robustness untested: this confirmatory run did not include active adversarial injection or red-team tool-description attacks; prior audits indicate such vectors can bypass naive defenses and remain a separate concern.
- Reproducibility gap for contamination check: the randomized holdout selection seed for the Mininet contamination check is absent from the labeled artifacts, preventing exact reconstruction of the holdout selection.
- Single repair budget: the GVR pipeline allowed at most one automated repair prompt per task; multi-step repair strategies, different repair budgets, or human-in-the-loop approvals were not exercised here.

VII. Conclusion

We preregistered and executed a sealed paired study comparing LLM-only generation to a deterministic GVR pipeline on 200 NetOps intent→configuration tasks using gpt-5-mini and deterministic_netops_validator_v5. Both pipelines produced validator-passing configurations for every task (n11 = 200; no discordant pairs), resulting in an observed paired difference of 0.0, exact McNemar p = 1.0, and bootstrap 95% CI = [0.0, 0.0]. The preregistered >=50% relative-reduction hypothesis could not be evaluated because baseline validator failures were absent. The artifact-grounded outcome clarifies that GVR's marginal value is workload dependent: when baseline validator-detectable failures are rare, automated repair yields no measured benefit for those validator-detected errors. Future studies should target workloads with nontrivial baseline failure rates, expand validator fidelity with richer emulation or real-device checks, evaluate multi-sample and multi-model policies, and explore multi-step or human-in-the-loop repair budgets to characterize practical operational gains. All execution artifacts and analysis outputs are archived for independent inspection under the preregistration id cited above.

Disclosure Statement

Disclosure (mandatory). This study executed under the autonomous post-lock preregistration cnsms2026-gvr-effectiveness-02. Declared model: gpt-5-mini (provider label: openai_responses) invoked via the hosted_netops_gvr_v1 adapter. The exact initial master prompt is archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196 add77f5acdf1582bb39b15ba). Topic constraints: the human Research Director limited the study to Generative AI and LLMs for NetOps focusing on safety/reliability of generated network changes. Frameworks and tools: orchestration used hosted_netops_gvr_v1 and the deterministic task generator deterministic_netops_task_generator_v5. Primary deterministic validator: deterministic_netops_validator_v5 (intent/constraint checks). A Mininet-based emulator was preregistered and executed as a secondary disagreement detector on a randomized 10% holdout (20 tasks); the artifact analysis/contamination_summary.csv shows zero disagreements for that holdout. Preregistration and freeze: the experimental plan (estimand, sample size n = 200, paired design, stopping rules, max model calls = 400, repair budget = 1 per task, shared-initial generation semantics) was frozen before any model calls. Autonomous execution and artifacts: the run executed autonomously under the frozen plan (start: 2026-08-30T11:50:34.530059Z; end: 2026-08-30T12:12:07.541650Z); model calls, prompts, seeds, validator traces, emulator traces, logs, and the artifact manifest are archived in the execution bundle. Model-call budget and usage: 200 model calls were used (one shared initial generation per task); up to 200 repair calls were allowed but none were applied because initial candidates passed the validator. Human involvement: the human Research Director selected the topic family and pre-lock constraints; no human manual editing or selective rewriting of technical manuscript content occurred after the autonomy lock. Deviations and restarts: no post-preregistration design changes or technical restarts occurred. Known reproducibility gap: the explicit randomized selection seed used to draw the Mininet 10% holdout is not present as a labeled artifact in the archive; this absence prevents exact deterministic reconstruction of the drawn holdout despite the contamination summary reporting no disagreements. The master prompt archive reference above is the immutable prompt required by the track.

References

[1] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, Xiaolei Wang, Yifan Du, Yushuo Chen, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Peiyu Liu, Yiwen Hu, Jian-Yun Nie, Ji-Rong Wen, "A Survey of Large Language Models", 2026, 10.1007/s11704-026-60308-3

[2] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?", 2024, 10.1145/3656296

[3] Oscar G. Lira, Oscar Maurício Caicedo Rendón, Nelson L. S. da Fonseca, "Large Language Models for Zero Touch Network Configuration Management", 2024, 10.1109/mcom.001.2400368

[4] Chirag Shah, Ryen W. White, Reid Andersen, Georg Buscher, Scott Counts, Sarkar Snigdha Sarathi Das, Ali Montazeralghaem, Sathish Manivannan, J. Neville, Nagu Rangan, Tara Safavi, Siddharth Suri, Mengting Wan, Leijie Wang, Longqi Yang, "Using Large Language Models to Generate, Validate, and Apply User Intent Taxonomies", 2025, 10.1145/3732294

[5] omar altawil, Dr. Mustafa Al-Fayoumi, "Configuration-Level Semantic Brittleness in Tool-Using LLM Agents: A Factor-Ranking Study of Persona, History, and Tool Definition", 2026, 10.2139/ssrn.7203631

[6] Mohammadreza Rashidi, "Measuring How LLM Tool Descriptions, Cross-Tool Ambiguity, and Action Chains Compose into Excessive Agency Exploits", 2026, 10.21203/rs.3.rs-10646209/v1

[7] Yoshihiro Ando, "Secure-by-Default Guardrails for MCP-Based Tool Use in Multi-Modal LLM Agents", 2026, 10.36227/techrxiv.177006109.97957916/v1

[8] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, Yarin Gal, "Detecting hallucinations in large language models using semantic entropy", 2024, 10.1038/s41586-024-07421-0

[9] Shuyin Ouyang, Jie M. Zhang, Mark Harman, Meng Wang, "An Empirical Study of the Non-Determinism of ChatGPT in Code Generation", 2024, 10.1145/3697010

[10] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1